

arm

Formal Verification

Lecture for ETIN40

Part 2

Lars Viklund

2026-10-12

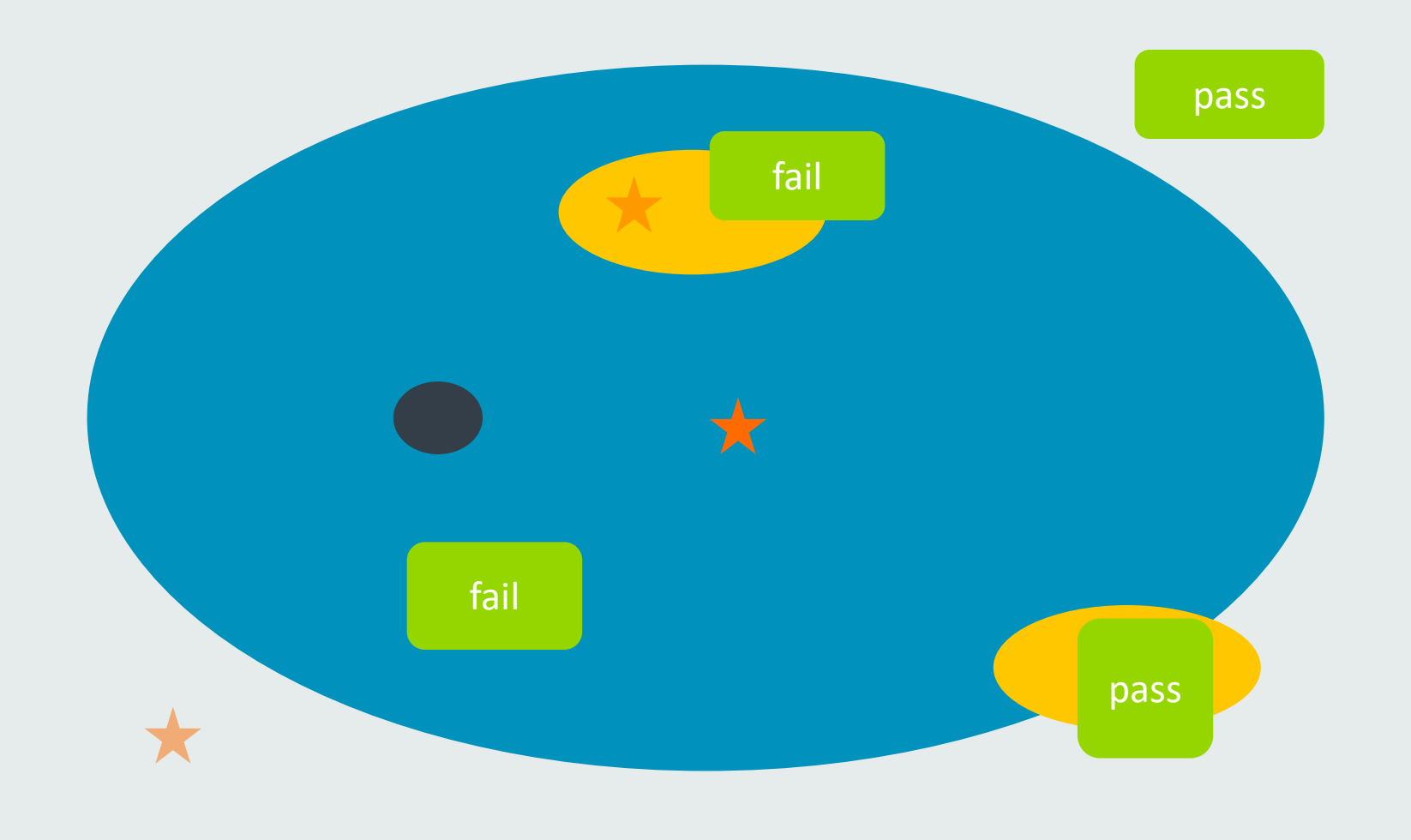
© 2026 Arm

Formal Property Verification (FPV)

- Method for proving whether a set of properties holds true for some design
- The inputs:
  - Design (RTL)
  - Properties to prove
  - Constraints
    - Assumptions, definitions of clock and reset, *etc.*
- Outputs:
  - For each assert property
    - Proved / Disproved / Inconclusive
    - Counterexample waveform if disproven
  - For each cover property
    - Coverable / Uncoverable / Inconclusive
    - Witness waveform if coverable

© 2026 Arm

State Space Analyzed by FPV



Reset States

Reachable States

Assumptions

Assertions

Cover properties

© 2026 Arm

FPV Applicability

- Type of Design
  - Control logic or data transport tends to be more formal friendly
- Sequential Depth
- Interface Types
- Block Specification
- Block Criticality and Risk
- Block Size

© 2026 Arm

FPV Metrics

- Verification Quality
  - Unbounded proofs, or potentially bounded proofs with justified depth
- Stimulus Coverage
  - Manually written or automatically generated cover properties to check for overconstraining
  - No vacuous proofs
  - Include assertions and assumptions in simulation when possible
- Cone of Influence
  - Check that RTL code is in the COI of some assertion
- Proof Core
  - Checks that RTL code is used in some proof
  - Subset of COI
- Mutation Coverage
  - The tool makes a change to the design and checks that some assertion then fails
- Mapping of requirements to properties

© 2026 Arm

Formal Apps

- Domain-specific
- Less formal expertise required
- Easier debugging

© 2026 Arm

Examples of Formal Apps

- Automatic Checks
- Connectivity Checking
- Coverage Unreachability Analysis
- X-propagation Verification
- Control and Status Register Verification
- Security Path Verification

© 2026 Arm

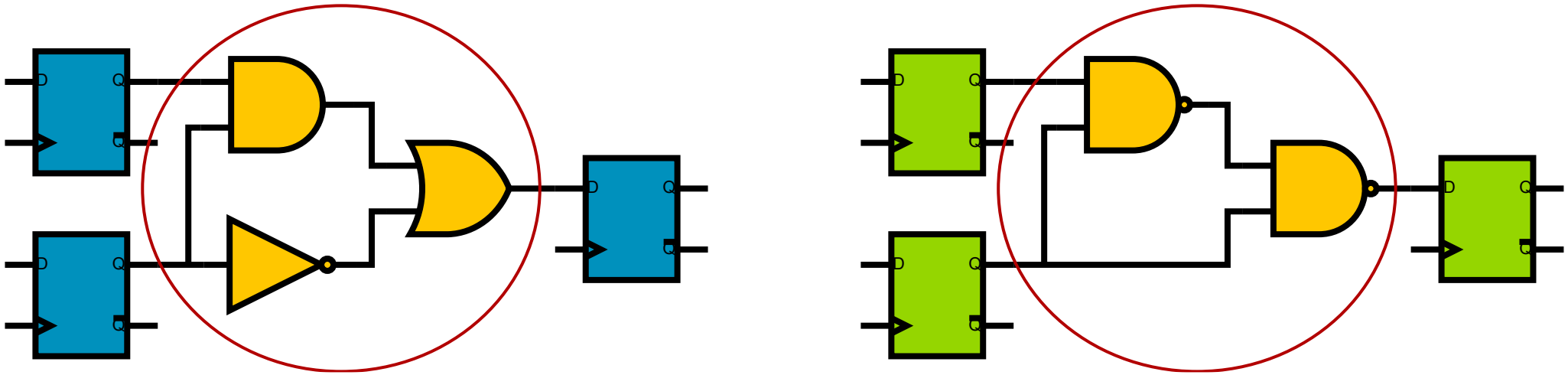
Formal Equivalence Checking

- Check whether two designs are equivalent
- Two models being compared
  - Specification
  - Implementation
- The specification will typically be more abstract
- Different kinds of equivalence:
  - Combinational equivalence
  - Sequential equivalence
  - Transactional equivalence

© 2026 Arm

Combinational Equivalence

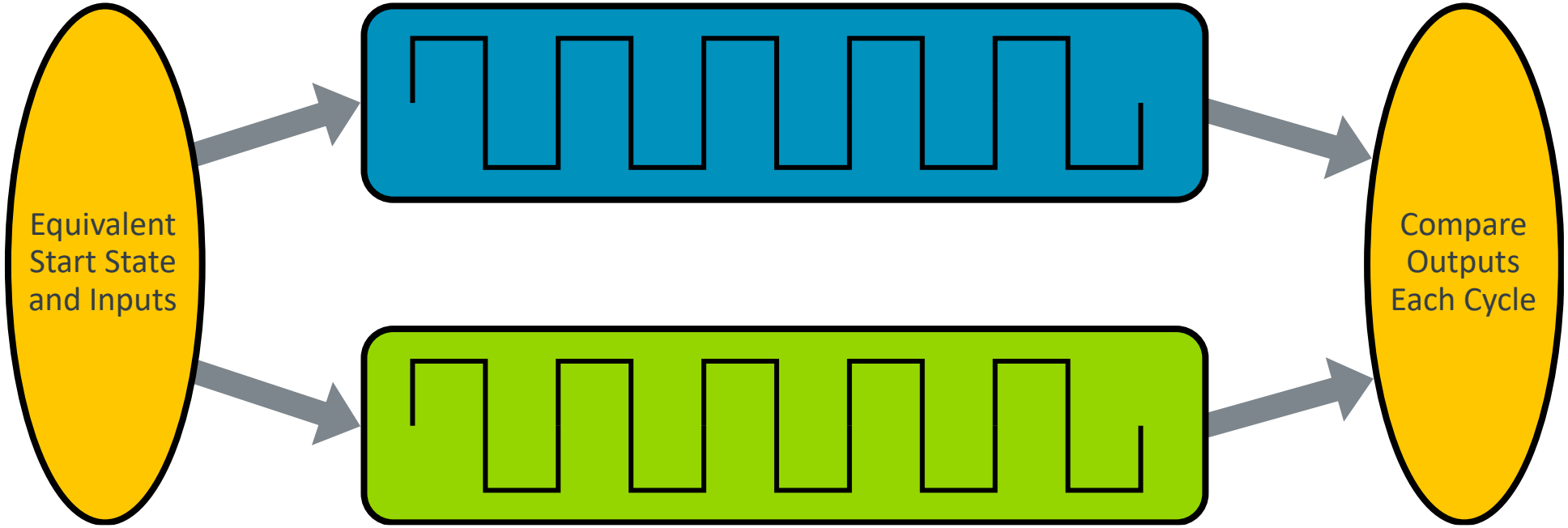
- Check that combinational logic between state elements is equivalent
  - 1-1 mapping of state elements



© 2026 Arm

Sequential Equivalence

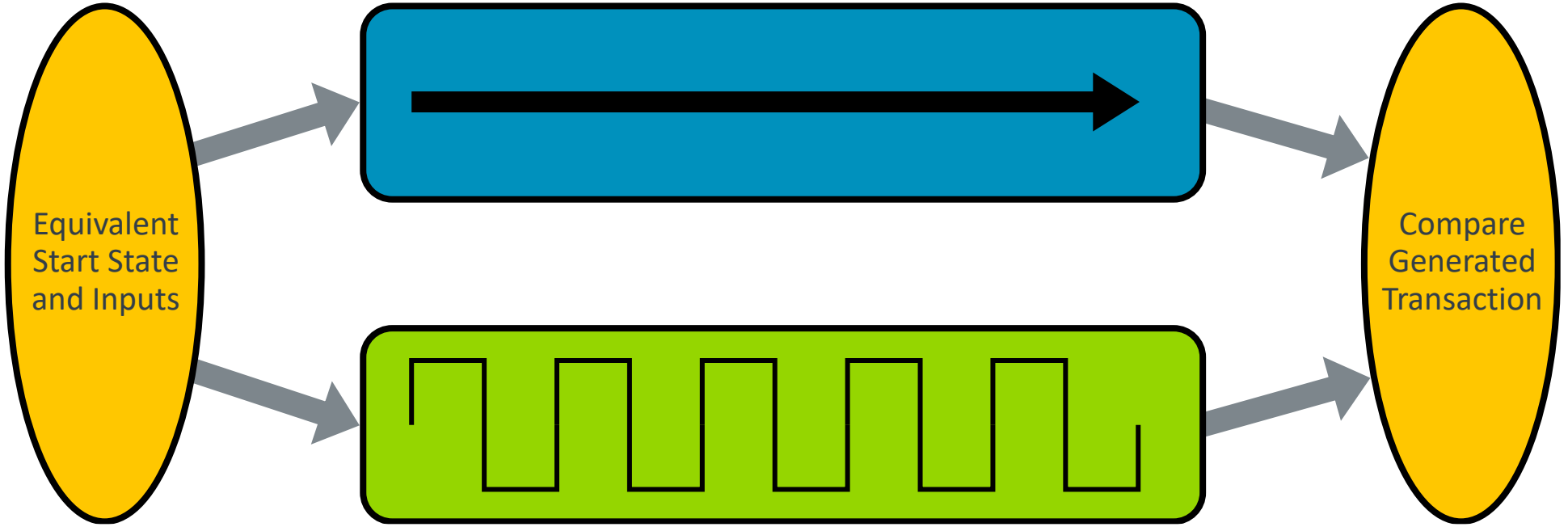
- Check that models produce the same outputs at the same times
  - Assuming equivalent initial states and identical inputs



© 2026 Arm
arm

Transactional Equivalence

- Check that models produce matching results for corresponding transactions



© 2026 Arm
arm

Equivalence Checking Use Cases

- RTL to Netlist
  - Result of logic synthesis
  - Manual changes to netlist
- Netlist to Netlist
  - Manual changes to netlist
- RTL to RTL
  - Parameterization
  - Timing fixes
  - Pipeline optimizations
  - Chicken bits
  - Clock gating
- C to RTL
  - Verification by comparing against an untimed reference model

© 2026 Arm

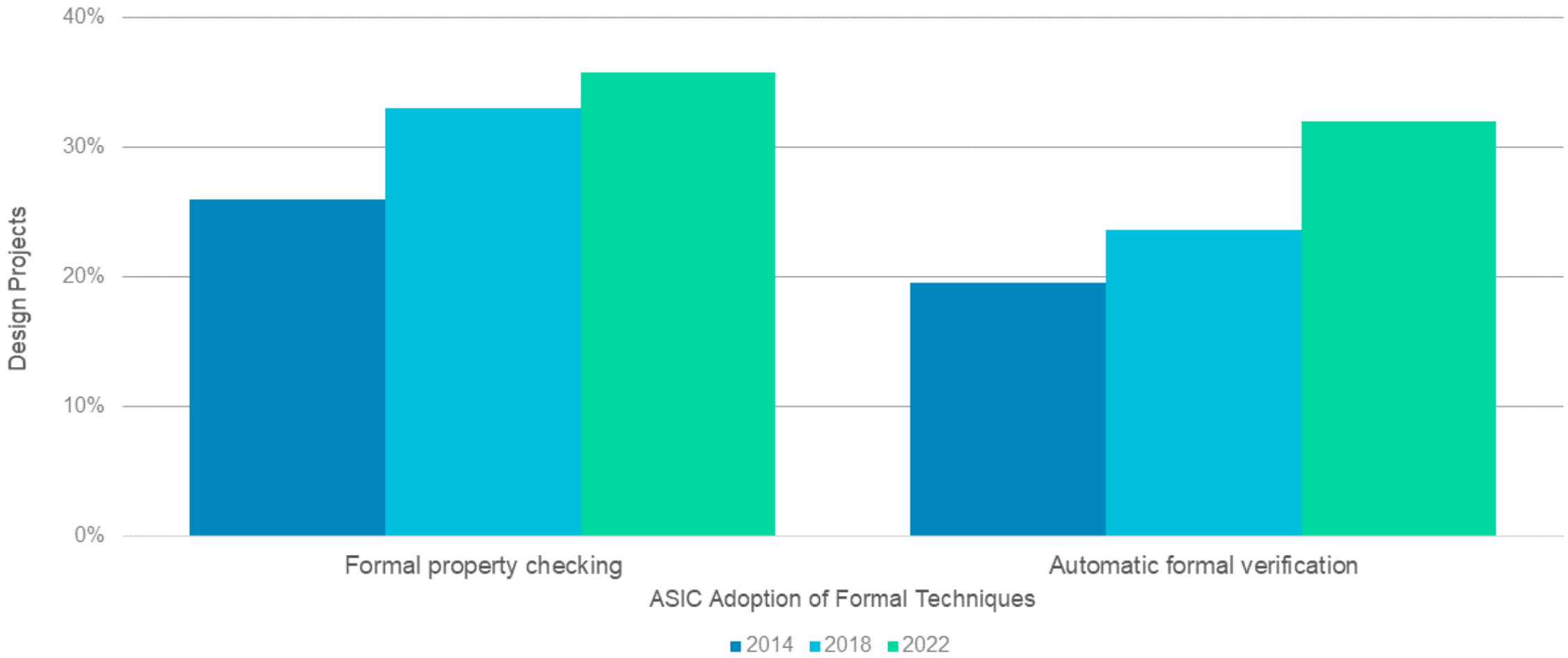
Handling Complexity

- Cut points
- Black boxing
- Case splitting
  - soft or hard
- Abstraction models
  - counter abstraction
  - memory abstraction
  - sequence abstraction
  - Wolper coloring

- Divide and conquer
- Assume-guarantee
- Property simplification
- Design reduction
- Generalizing properties using symbolic variables
- Initial value abstraction
- Engine tuning

© 2026 Arm

ASIC adoption of formal techniques



Source: Wilson Research Group and Siemens EDA, 2022 Functional Verification Study

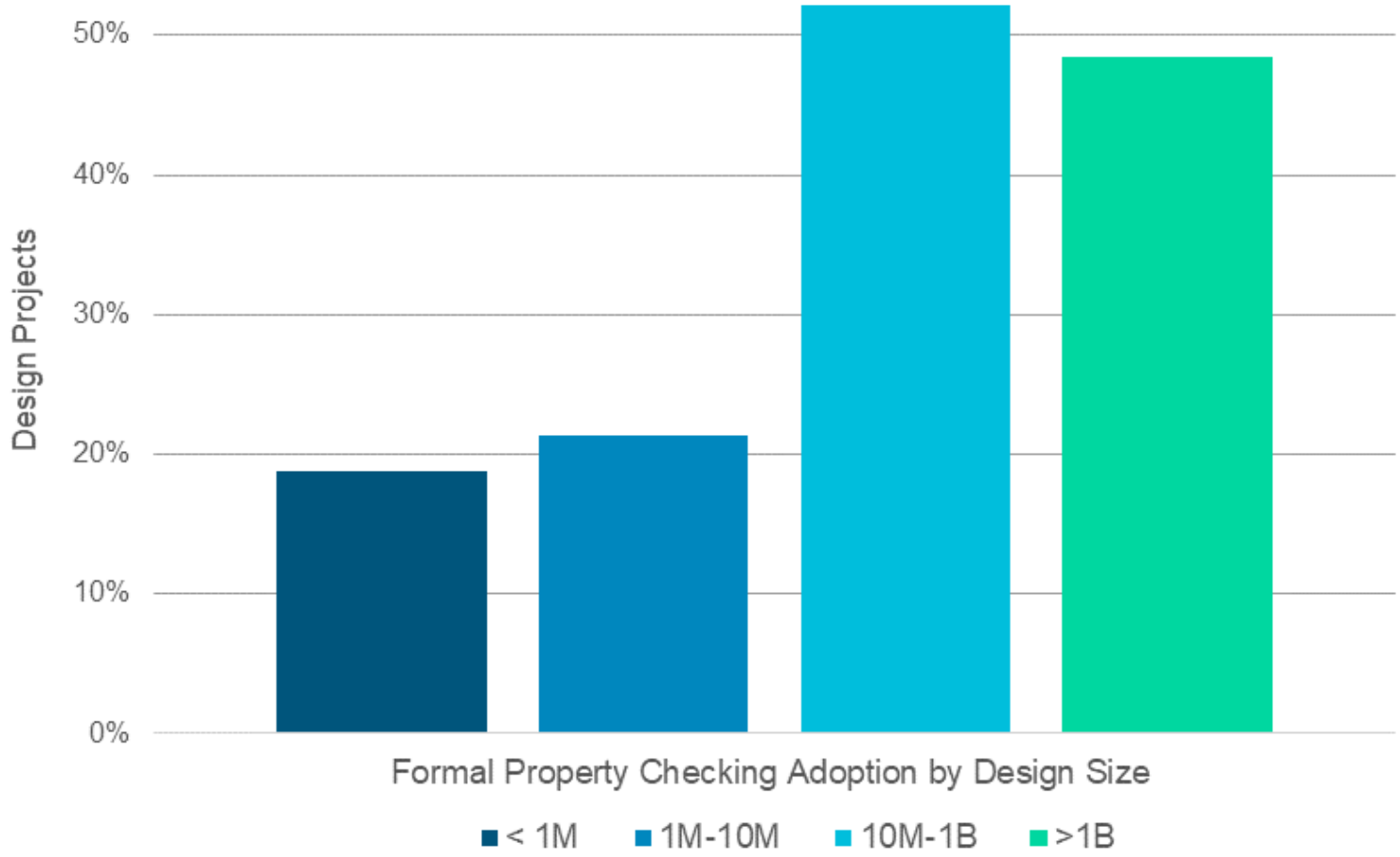
SIEMENS

ASIC adoption of formal property checking by design size

Gates of logic and datapath excluding memories

~50%

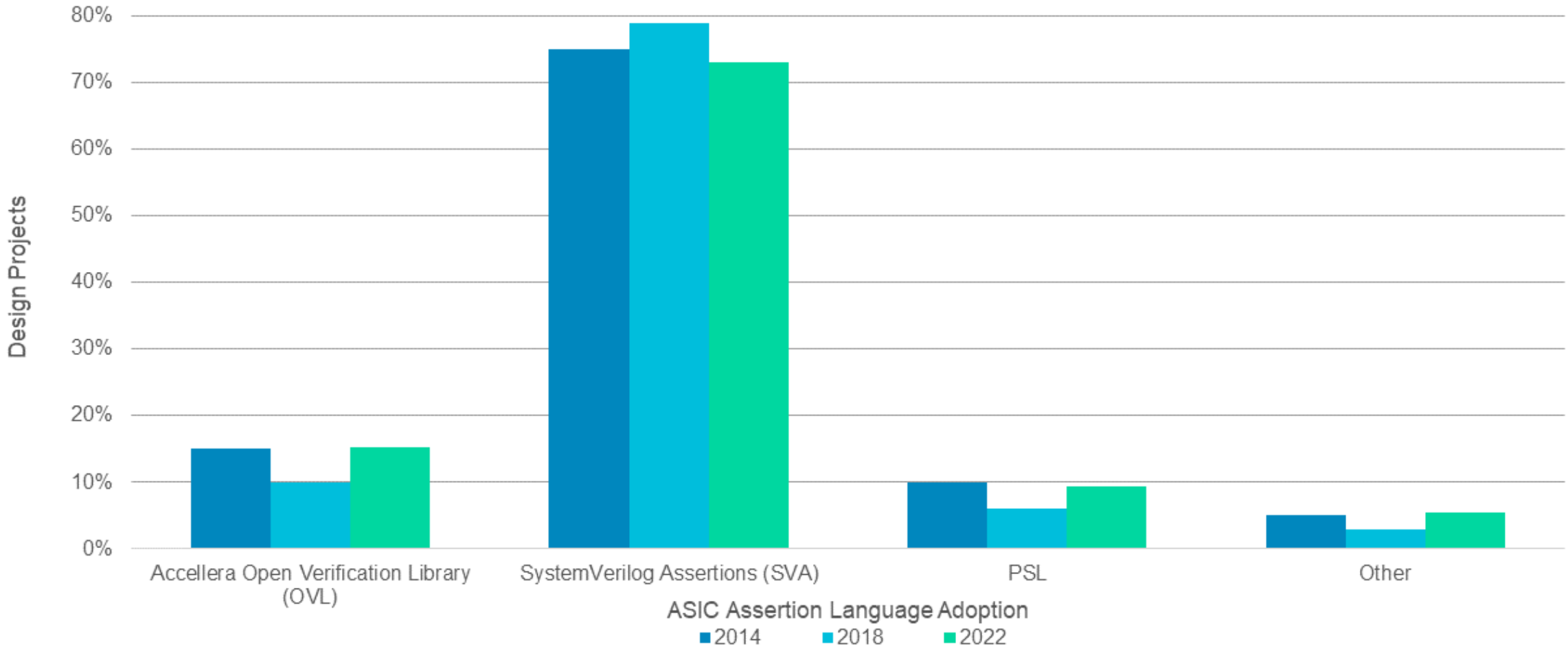
Adoption of formal property checking on designs greater than 10M gates.



Source: Wilson Research Group and Siemens EDA, 2022 Functional Verification Study

SIEMENS

ASIC assertion language adoption



Source: Wilson Research Group and Siemens EDA, 2022 Functional Verification Study

* Multiple replies possible

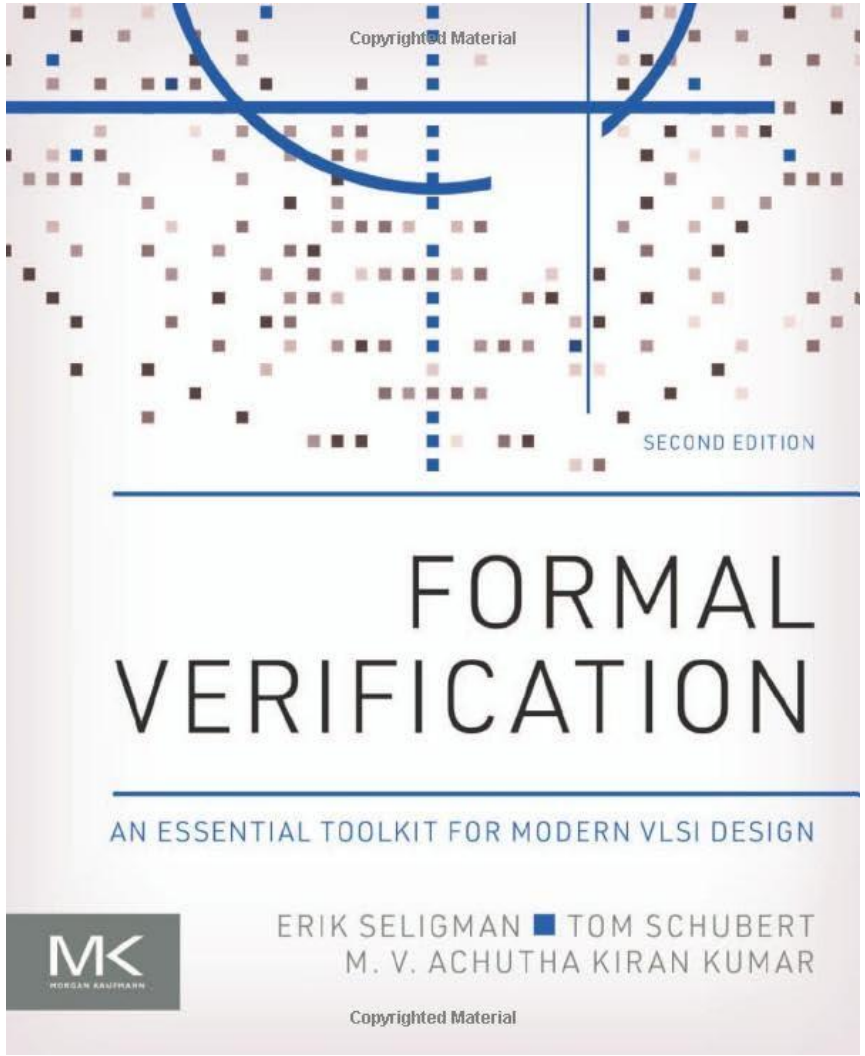
SIEMENS

arm

Live Demo

© 2026 Arm

Want to Learn More?



© 2026 Arm

Summary

- Formal hardware verification is mainstream
  - But simulation is still the go-to method
- Powerful tool to complement other verification techniques
  - Need to understand strengths and weaknesses
- Tool capacity is increasing, but dealing with complexity is a major issue
- Formal apps lower the bar for getting started

© 2026 Arm



Thank You
Danke
Gracias
Grazie
谢谢
ありがとう
Asante
Merci
감사합니다
धन्यवाद
Kiitos
شكرًا
ধন্যবাদ
תודה
ధన్యవాదములు

© 2026 Arm



The Arm trademarks featured in this presentation are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved. All other marks featured may be trademarks of their respective owners.

www.arm.com/company/policies/trademarks

© 2026 Arm